Встроенный оператор && обладает коротким замыканием и вычисляет левый операнд раньше правого. Если перегрузить operator&&, выражение станет обычным вызовом функции operator&&(a, b). Тогда оба аргумента будут вычислены до вызова функции, и короткое замыкание исчезнет. Порядок вычисления аргументов функции также не гарантирует, что левый операнд будет вычислен первым.

Это опасно в случаях из разряда:

```
if (pointer != nullptr && pointer->value() > 0) { ... }
```

При встроенном && разыменования при nullptr не будет, а при перегруженном может произойти.

Поэтому вместо перегрузки && лучше использовать explicit operator bool() или именованный метод вроде isValid().